

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG



BÀI TẬP LỚN

Học phần: Thiết kế và xây dựng phần mềm
(Mã học phần: IT4490)

Đề tài:

AIMS – Cửa hàng trực tuyến

Mã lớp học : 154041
Giảng viên hướng dẫn : TS. Nguyễn Thị Thu Trang
Danh sách sinh viên thực hiện:

STT	Họ và tên	Mã sinh viên
1	Nguyễn Trung Chiến	20215320
2	Nguyễn Mạnh Cường	20204524
3	Vũ Ngọc Anh	20215313
4	Bùi Minh Hải Đắc	20204525
5	Trần Đức Chính	20210123

Hà Nội, Tháng 12 năm 2024

DANH MỤC BẢNG.....	3
DANH MỤC HÌNH ẢNH.....	4
LỜI NÓI ĐẦU	5
Chương 1: Phân tích yêu cầu.....	6
1. Mô tả yêu cầu bài toán	6
2. Xác định tác nhân.....	10
3. Biểu đồ use case tổng quan	11
4. Các chức năng thực hiện	11
Chương 2: Phân tích thiết kế	12
1. Các chức năng chính	12
1.1. Use case Place order – Đặt hàng	12
1.2. Use case Pay order – Thanh toán đơn hàng	12
2. Biểu đồ lớp thiết kế	13
2.1. Biểu đồ lớp thiết kế tổng quan	13
2.2. Biểu đồ lớp thiết kế cho use case đặt hàng	14
2.3. Biểu đồ lớp thiết kế cho hệ thống thanh toán	14
3. Thiết kế cơ sở dữ liệu.....	15
3.1. Phân tích các thực thể	15
3.2. Thiết kế cơ sở dữ liệu.....	15
Chương 3: Xây dựng	17
1. Xây dựng frontend bằng ReactJS.....	17
2. Xây dựng backend bằng Spring Boot	17
3. Cơ sở dữ liệu MySQL	17
Chương 4: Kiểm thử đơn vị	19
1. Thiết kế test cases.....	19
2. Kết quả kiểm thử.....	21
Chương 5: Đánh giá.....	24
1. Cohesion, coupling.....	24
1.1. Coupling.....	24

Thiết kế và xây dựng phần mềm

1.2. Cohesion.....	25
2. Nguyên tắc thiết kế SOLID.....	25
2.1. Single Responsibility Principle	25
2.2. Open-Closed Principle	25
2.3. Liskov Substitution Principle.....	25
2.4. Interface Segregation Principle	26
2.5. Dependency Inversion Principle	26
3. Nguyên tắc thiết kế GRASP.....	26
3.1. Information Expert.....	26
3.2. Creator.....	27
3.3. Controller	27
3.4. Low Coupling	27
3.5. High Cohesion.....	28
3.6. Polymorphism	28
3.7. Pure Fabrication	28
3.8. Indirection	28
3.9. Protected Variations.....	29
3.10. General Observations.....	29
3.11. Recommendations.....	29
Chương 6: Mẫu thiết kế Design Pattern.....	31
1. Singleton Pattern	31
2. Factory Method Pattern.....	31
3. Template Method Pattern.....	31
4. Adapter Pattern.....	32
5. Builder Pattern	32
6. Strategy Pattern	33
Chương 7: Kết luận và hướng phát triển	34
TÀI LIỆU THAM KHẢO	35

DANH MỤC BẢNG

Bảng 1. Thiết kế test case validateName().....	19
Bảng 2. Thiết kế test cases validatePhoneNumber()	19
Bảng 3. Thiết kế test cases validateAddress()	20
Bảng 4. Thiết kế test cases calcShippingFees()	20
Bảng 5. Common coupling	24
Bảng 6. Stamp coupling	24

DANH MỤC HÌNH ẢNH

Hình 1 Biểu đồ use case tổng quan	11
Hình 2. Biểu đồ tuần tự use case Place order - đặt hàng.....	12
Hình 3. Biểu đồ tuần tự use case Thanh toán đơn hàng.....	13
Hình 4. Biểu đồ lớp thiết kế tổng quan	13
Hình 5. Biểu đồ lớp thiết kế cho use case Place order - đặt hàng.....	14
Hình 6. Biểu đồ lớp thiết kế cho use case Pay order - thanh toán đơn hàng .	14
Hình 7. Sơ đồ thực thể liên kết phân tích.....	15
Hình 8. Sơ đồ thực thể liên kết	16
Hình 9. Kết quả test 01.....	21
Hình 10. Kết quả test 02.....	21
Hình 11. Kết quả test 03.....	22
Hình 12. Kết quả test 04.....	23
Hình 13. Strategy pattern	33

LỜI NÓI ĐẦU

Trong bối cảnh thương mại điện tử phát triển mạnh mẽ, nhu cầu về các nền tảng mua bán trực tuyến chuyên biệt ngày càng gia tăng. Báo cáo này trình bày chi tiết về thiết kế và phát triển AIMS (An Internet Media Store) - một phần mềm thương mại điện tử desktop được xây dựng nhằm đáp ứng nhu cầu mua bán các sản phẩm media một cách hiệu quả và chuyên nghiệp.

AIMS được thiết kế với mục tiêu tạo ra một nền tảng ổn định, có khả năng phục vụ đồng thời nhiều người dùng, đồng thời đảm bảo tính dễ sử dụng cho người mới. Hệ thống hướng đến việc cung cấp một môi trường mua bán trực tuyến an toàn và tiện lợi cho các sản phẩm media vật lý như sách, CD, đĩa than (LP) và DVD.

Báo cáo này sẽ mô tả chi tiết các yêu cầu kỹ thuật, kiến trúc hệ thống, và các tính năng chính của AIMS, bao gồm:

- Quản lý sản phẩm và kho hàng
- Quy trình đặt hàng và thanh toán
- Hệ thống giao hàng thông thường và giao hàng nhanh
- Tích hợp cổng thanh toán VNPay
- Quản lý người dùng và phân quyền

Với các tính năng được thiết kế cẩn thận và yêu cầu hiệu năng rõ ràng, AIMS hứa hẹn sẽ là một giải pháp phần mềm đáp ứng được nhu cầu của cả người bán và người mua trong lĩnh vực kinh doanh sản phẩm media.

Chương 1: Phân tích yêu cầu

1. Mô tả yêu cầu bài toán

AIMS: Một cửa hàng phương tiện truyền thông trực tuyến

Dự án AIMS là một phần mềm thương mại điện tử dành cho máy tính để bàn, hoạt động 24/7, cho phép người dùng mới dễ dàng làm quen. Phần mềm này có thể phục vụ đồng thời lên đến 1.000 khách hàng mà không làm giảm hiệu suất đáng kể và có thể hoạt động liên tục trong 300 giờ mà không gặp lỗi. Ngoài ra, phần mềm có thể tiếp tục hoạt động bình thường trong vòng tối đa 1 giờ sau khi xảy ra sự cố. Thời gian phản hồi tối đa của phần mềm là 2 giây trong điều kiện bình thường hoặc 5 giây trong giờ cao điểm.

Trong AIMS, quản lý sản phẩm có thể thêm, xem, chỉnh sửa hoặc xóa bất kỳ sản phẩm nào. Tuy nhiên, quản lý sản phẩm chỉ có thể thêm hoặc chỉnh sửa một sản phẩm tại một thời điểm, nhưng họ có thể xóa tối đa 10 sản phẩm cùng lúc. Ngoài ra, quản lý sản phẩm không thể xóa hoặc cập nhật hơn 30 sản phẩm vì lý do bảo mật, nhưng họ có thể thêm số lượng sản phẩm không giới hạn trong một ngày.

Khi muốn thêm sản phẩm để bán, quản lý sản phẩm cần cung cấp thông tin bắt buộc theo yêu cầu của phần mềm. Đối với mỗi sản phẩm phương tiện, quản lý sản phẩm cần cung cấp tiêu đề, danh mục, giá trị và giá hiện tại. Giá trị và giá của sản phẩm mà họ nhập không bao gồm thuế giá trị gia tăng (VAT) 10%. Tùy thuộc vào loại sản phẩm, quản lý sản phẩm cần cung cấp các thông tin sau:

Sách yêu cầu thông tin về tác giả, loại bìa (bìa mềm hoặc bìa cứng), nhà xuất bản và ngày xuất bản; thông tin bổ sung như số trang, ngôn ngữ và thể loại có thể được bao gồm.

Đĩa CD, bao gồm các bộ sưu tập âm nhạc hoặc album, yêu cầu thông tin về nghệ sĩ, hãng thu âm, danh sách bài hát và thể loại; thông tin bổ sung như ngày phát hành có thể được bao gồm.

Đĩa LP có các yêu cầu thông tin tương tự như CD.

Đĩa DVD yêu cầu thông tin về loại đĩa (Blu-ray, HD-DVD), đạo diễn, thời lượng, hãng phim, ngôn ngữ và phụ đề; thông tin bổ sung như ngày phát hành và thể loại có thể được bao gồm.

Hiện tại, phần mềm chỉ cho phép mua và bán các sản phẩm phương tiện vật lý. Đối với những loại sản phẩm này, yêu cầu cung cấp mã vạch, mô tả sản phẩm (ví dụ: mới, đã qua sử dụng, màu sắc chính, điều kiện trả lại), số lượng, ngày nhập kho, kích thước và trọng lượng sản phẩm. Các sản phẩm vật lý có thể là sách, đĩa CD, đĩa LP hoặc đĩa DVD.

Thiết kế và xây dựng phần mềm

Giá của sản phẩm có thể thay đổi tùy thuộc vào nhu cầu thị trường. Quản lý sản phẩm có thể cập nhật giá của một sản phẩm tối đa hai lần mỗi ngày. Tuy nhiên, giá của sản phẩm phải luôn nằm trong khoảng từ 30% đến 150% giá trị của sản phẩm để tránh lạm phát giá hoặc bán phá giá.

Phần mềm sẽ lưu trữ lịch sử của các thao tác thêm, chỉnh sửa và xóa sản phẩm và sẽ thông báo cho quản lý sản phẩm nếu có bất kỳ thao tác nào không hợp lệ. Ví dụ, khi thông tin nhập vào không tuân thủ quy tắc hoặc định dạng ngày không đúng.

Quản trị viên có thể tạo người dùng mới, xem thông tin người dùng, cập nhật thông tin người dùng, xóa người dùng và đặt lại mật khẩu. Họ có thể chặn hoặc mở khóa người dùng khi cần thiết. Phần mềm sẽ tự động gửi email thông báo cho người dùng về các hành động quản trị. Quản trị viên có thể thiết lập hoặc thay đổi vai trò của người dùng. Mỗi người dùng có thể có nhiều vai trò, chẳng hạn như quản trị viên hoặc quản lý sản phẩm. Quản trị viên và quản lý sản phẩm cần đăng nhập để truy cập các tính năng tương ứng với vai trò của họ. Họ cũng có thể thay đổi mật khẩu.

Đối với khách hàng, khi khởi động phần mềm, một danh sách 20 sản phẩm ngẫu nhiên sẽ được hiển thị trên mỗi trang. Để tìm kiếm sản phẩm, khách hàng sử dụng các thuộc tính của sản phẩm để tìm kiếm. Phần mềm sẽ hiển thị 20 sản phẩm liên quan trên mỗi trang tìm kiếm. Ngoài ra, khách hàng có thể sắp xếp sản phẩm theo giá. Khách hàng có thể xem thông tin chi tiết của mỗi sản phẩm khi chọn sản phẩm tương ứng trong danh sách sản phẩm. Khách hàng có thể thêm sản phẩm với số lượng tương ứng vào giỏ hàng hiện tại từ danh sách sản phẩm hoặc màn hình chi tiết sản phẩm.

Khi khách hàng yêu cầu xem giỏ hàng, phần mềm sẽ hiển thị thông tin giỏ hàng, bao gồm tổng giá sản phẩm chưa bao gồm VAT, danh sách sản phẩm với thông tin sản phẩm (tên sản phẩm, số lượng và giá). Đồng thời, phần mềm cũng sẽ thông báo cho khách hàng nếu số lượng tồn kho của bất kỳ sản phẩm nào không đủ và sẽ hiển thị số lượng sản phẩm bị thiếu. Khi thay đổi ý định, khách hàng cũng có thể xóa sản phẩm khỏi giỏ hàng hoặc thay đổi số lượng sản phẩm trong giỏ.

Ngoài ra, trong mỗi phiên làm việc với phần mềm, chỉ có một giỏ hàng duy nhất và giỏ hàng sẽ được làm trống sau khi thanh toán thành công đơn hàng.

Để đặt hàng thành công, khách hàng cần tiến hành đặt hàng và thanh toán. Trong phạm vi khóa học, khách hàng không cần đăng nhập để đặt hàng. Khi đặt hàng, khách hàng cần cung cấp thông tin giao hàng (bao gồm tên người nhận, email, số điện thoại, tỉnh/thành phố giao hàng và địa chỉ giao hàng) cũng như thông tin thanh toán.

Trước khi đặt hàng, khách hàng cần xem lại giỏ hàng và chọn sản phẩm muốn mua. Từ đây, khách hàng có thể yêu cầu đặt hàng. Mỗi khi khách hàng yêu cầu đặt hàng, phần mềm sẽ luôn kiểm tra xem số lượng hàng tồn kho có đủ để cung cấp cho khách hàng hay không. Phần mềm sẽ yêu cầu khách hàng cập nhật giỏ hàng nếu số lượng hàng tồn kho không đủ và sẽ hiển thị số lượng tồn kho của mỗi sản phẩm không đủ. Sau khi cập nhật giỏ hàng, khách hàng có thể yêu cầu đặt hàng lại.

Thiết kế và xây dựng phần mềm

Tiếp theo, phần mềm sẽ yêu cầu khách hàng thiết lập thông tin giao hàng. Trong quá trình nhập thông tin giao hàng, khách hàng vẫn có thể thấy các sản phẩm và thông tin như trong giỏ hàng. Lúc này, phần mềm sẽ kiểm tra thông tin đầu vào và yêu cầu khách hàng cập nhật nếu có trường thông tin nào bị bỏ trống hoặc thông tin không hợp lệ. Mỗi khi khách hàng nhập hoặc cập nhật địa chỉ giao hàng, phần mềm sẽ tính toán (hoặc tính lại) và hiển thị phí giao hàng. Sau đó, khách hàng có thể tiến hành thanh toán đơn hàng. Lúc này, phần mềm sẽ hiển thị và lưu tạm thời thông tin hóa đơn, bao gồm danh sách sản phẩm trong giỏ hàng, số lượng, giá sản phẩm, tổng giá sản phẩm chưa bao gồm VAT, tổng giá sản phẩm đã bao gồm VAT, phí giao hàng và tổng số tiền cần thanh toán. Tổng số tiền mà khách hàng cần thanh toán bao gồm tổng giá sản phẩm đã bao gồm VAT và phí giao hàng. Sau đó, phần mềm sẽ cho phép khách hàng chọn phương thức thanh toán. Trong phạm vi khóa học, chỉ có một tùy chọn thanh toán qua thẻ tín dụng.

Tuy nhiên, trong tương lai, chúng tôi có thể mở rộng sang các phương thức thanh toán khác. Phần mềm AIMS kết nối với VNPay để xử lý thanh toán qua thẻ tín dụng. Khách hàng cần cung cấp tất cả các thông tin và bước cần thiết theo yêu cầu của VNPay để hoàn tất giao dịch thành công.

Khách hàng có tùy chọn chọn giao hàng nhanh. Giao hàng nhanh cho phép khách hàng nhận hàng trong khoảng thời gian đã thỏa thuận trước đó là 2 giờ. Hiện tại, giao hàng nhanh chỉ khả dụng cho các địa chỉ trong nội thành Hà Nội (chỉ áp dụng với các quận). Nếu khách hàng muốn chọn giao hàng nhanh, phần mềm sẽ kiểm tra xem địa chỉ giao hàng có hỗ trợ dịch vụ này và liệu các sản phẩm có đủ điều kiện hay không. Nếu không có sản phẩm nào đủ điều kiện hoặc địa chỉ giao hàng không hỗ trợ giao hàng nhanh, phần mềm sẽ yêu cầu khách hàng cập nhật thông tin giao hàng hoặc phương thức giao hàng. Trong các trường hợp mà cả sản phẩm và địa chỉ giao hàng đều hỗ trợ giao hàng nhanh, phần mềm sẽ yêu cầu khách hàng cung cấp thêm thông tin về giao hàng nhanh (thời gian giao hàng nhanh, hướng dẫn giao hàng) và đồng thời cung cấp thông tin như sau:

Nếu chỉ có một số sản phẩm đủ điều kiện giao hàng nhanh, những sản phẩm này sẽ được giao cùng nhau đến địa điểm giao hàng nhanh vào thời gian đã thỏa thuận. Những sản phẩm không đủ điều kiện giao hàng nhanh sẽ được giao như bình thường. Phí giao hàng sẽ được tính toán và hiển thị riêng cho các sản phẩm giao hàng bình thường (nếu có) và sản phẩm giao hàng nhanh.

Nếu tất cả sản phẩm trong đơn hàng đủ điều kiện giao hàng nhanh, chúng sẽ được giao cùng nhau đến địa điểm giao hàng nhanh vào thời gian đã thỏa thuận. Phí giao hàng sẽ được tính toán và hiển thị cho nhóm sản phẩm này.

Khách hàng có thể điều chỉnh phương thức giao hàng hoặc các sản phẩm họ muốn mua nếu cần thiết. Phần mềm sẽ tính toán lại phí giao hàng và cập nhật hóa đơn tương ứng. Phí giao hàng phụ thuộc vào trọng lượng của sản phẩm và địa điểm giao hàng.

Thiết kế và xây dựng phần mềm

Phí vận chuyển không chịu thuế.

Các đơn hàng có tổng giá trị sản phẩm vượt quá 100.000 VND sẽ được miễn phí vận chuyển, tối đa là 25.000 VND mỗi đơn hàng. Điều này không áp dụng cho các sản phẩm giao hàng nhanh. Do đó, khách hàng luôn phải trả phí vận chuyển riêng cho các sản phẩm giao hàng nhanh. Tổng giá trị đơn hàng để đủ điều kiện miễn phí vận chuyển không bao gồm các sản phẩm giao hàng nhanh.

Phí vận chuyển được tính dựa trên trọng lượng của sản phẩm nặng nhất.

Đối với khách hàng ở nội thành Hà Nội hoặc TP. Hồ Chí Minh, giá ban đầu cho 3kg đầu tiên là 22.000 VND.

Đối với khách hàng ở các khu vực khác của Việt Nam, giá ban đầu cho 0,5kg đầu tiên là 30.000 VND.

Mỗi 0,5kg tiếp theo sẽ tính thêm phí 2.500 VND.

Trong trường hợp chọn giao hàng nhanh, khách hàng sẽ bị tính thêm phí 10.000 VND cho mỗi sản phẩm giao hàng nhanh.

Sau khi thanh toán thành công, phần mềm sẽ hiển thị thông tin chung về đơn hàng (tên khách hàng, số điện thoại, địa chỉ giao hàng, tỉnh/thành phố, tổng số tiền) và thông tin giao dịch (ID giao dịch, nội dung giao dịch và thời gian giao dịch). Đơn hàng sẽ ở trạng thái chờ xử lý và phần mềm sẽ gửi thông tin hóa đơn và giao dịch thanh toán đến email của khách hàng. Lưu ý rằng khách hàng có thể quay lại bất kỳ bước nào hoặc thoát khỏi phần mềm trong quá trình đặt hàng. Cuối cùng, phần mềm ghi lại thông tin giao dịch thanh toán và đơn hàng đã thanh toán thành công.

Khách hàng có thể xem thông tin đơn hàng hoặc hủy đơn hàng bằng các liên kết được gửi trong email về đơn hàng và giao dịch. Khách hàng có thể xem tất cả thông tin về đơn hàng, bao gồm hóa đơn, thông tin giao hàng và thông tin giao dịch thanh toán. Khách hàng có thể chọn hủy đơn hàng khi xem thông tin đơn hàng trước khi đơn hàng được phê duyệt. Sau khi khách hàng xác nhận hủy đơn hàng, toàn bộ số tiền sẽ được hoàn lại qua phương thức thanh toán đã sử dụng cho đơn hàng này thông qua VNPay.

Các đơn hàng trong trạng thái chờ xử lý sẽ được quản lý sản phẩm xem xét hoặc từ chối. Quản lý sản phẩm có thể thấy 30 đơn hàng chờ xử lý trên mỗi trang. Từ đó, họ có thể chọn một đơn hàng cụ thể để xem chi tiết và phê duyệt hoặc từ chối đơn hàng ngay cả khi có đủ sản phẩm trong kho. Có nhiều lý do khác nhau như không thể giao hàng, hết hàng trong khi khách hàng đang thanh toán hoặc đơn giản là không thể tìm thấy sản phẩm trong kho. Tuy nhiên, phần mềm sẽ từ chối chấp nhận đơn hàng nếu không đủ sản phẩm trong kho.

Thiết kế và xây dựng phần mềm

Trong phạm vi khóa học, sinh viên kết nối với VNPay Sandbox cho bất kỳ giao dịch thanh toán hoặc hoàn tiền nào. Một số thông tin và đặc tả về cổng thanh toán có thể được tìm thấy tại:

Bản demo tại: <https://sandbox.vnpayment.vn/apis/vnpay-demo/>

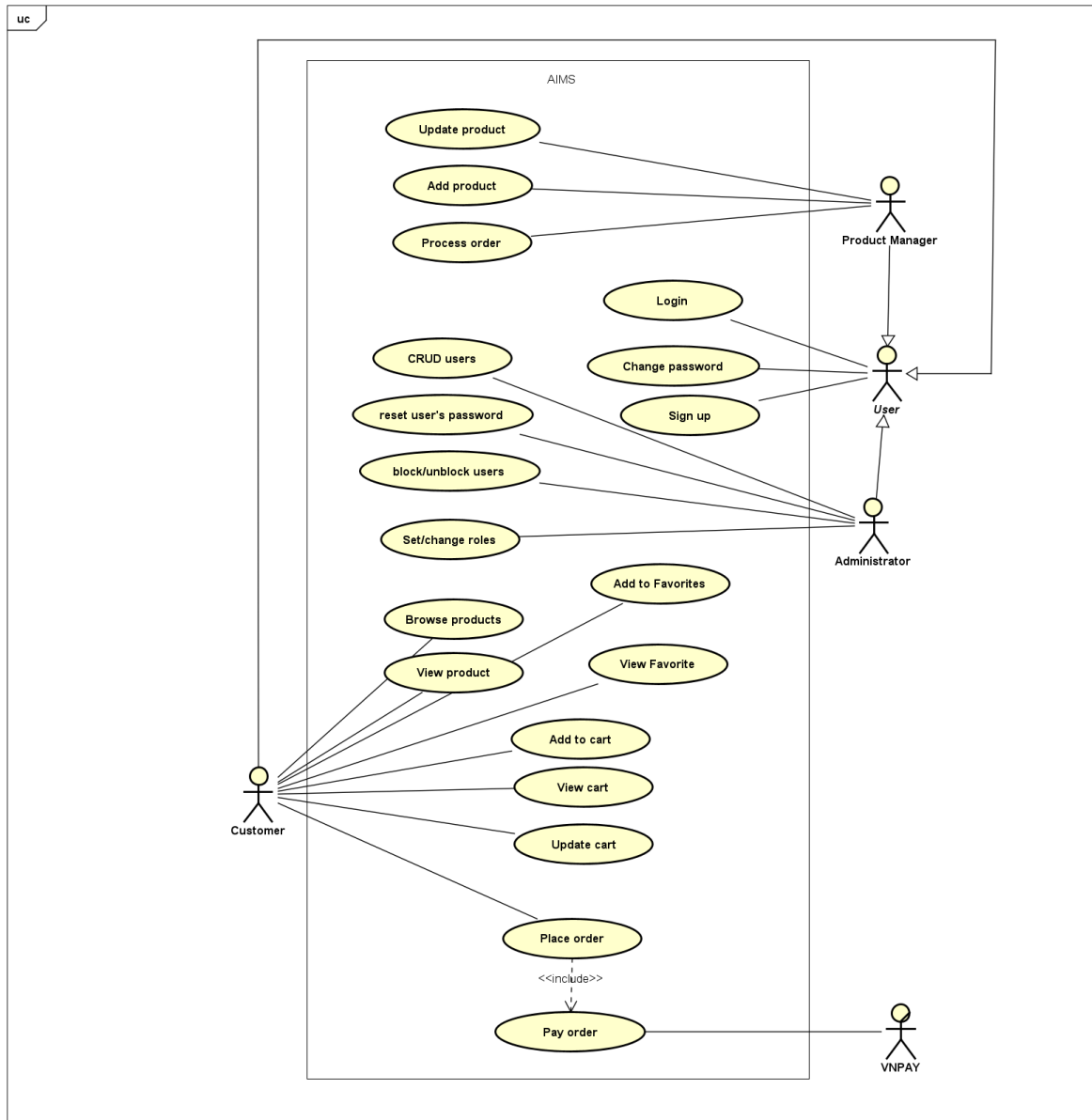
Đặc tả API thanh toán: <https://sandbox.vnpayment.vn/apis/docs/thanh-toanpay/pay.html>

Đặc tả API truy vấn và hoàn tiền: <https://sandbox.vnpayment.vn/apis/docs/truy-vanhoan-tien/querydr&refund.html>

2. Xác định tác nhân

STT	Tên tác nhân	Chức năng
1	Khách hàng chưa đăng nhập	Xem danh sách sản phẩm Xem thông tin chi tiết sản phẩm Đăng ký Đăng nhập Tìm kiếm sản phẩm
2	Khách hàng đã đăng nhập	Xem danh sách sản phẩm Xem thông tin chi tiết sản phẩm Tìm kiếm sản phẩm Đặt mua, quản lý đơn hàng Xem đơn hàng Thanh toán
3	Người quản lý sản phẩm	Thêm sản phẩm Cập nhật sản phẩm Xử lý các đơn hàng
4	Người quản trị	Thêm/sửa/xóa người dùng Đổi mật khẩu người dùng Chặn/mở chặn người dùng Gán/thay đổi vai trò người dùng

3. Biểu đồ use case tổng quan



Hình 1 Biểu đồ use case tổng quan

4. Các chức năng thực hiện

Các chức năng thực hiện bao gồm toàn bộ quy trình để tiến đến đặt và thanh toán thành công một đơn hàng. Cụ thể sẽ bao gồm: Sign up, Login, View products, Browse products, Add to cart, View cart, Place order, Pay order. Ngoài ra, nhóm tiến hành triển khai thêm tính năng mới: Add to favorites và View favorites – Thêm sản phẩm vào danh sách yêu thích và xem danh sách yêu thích.

Các thông tin chi tiết về phân tích yêu cầu bài toán được miêu tả trong tài liệu đặc tả yêu cầu (SRS) của nhóm

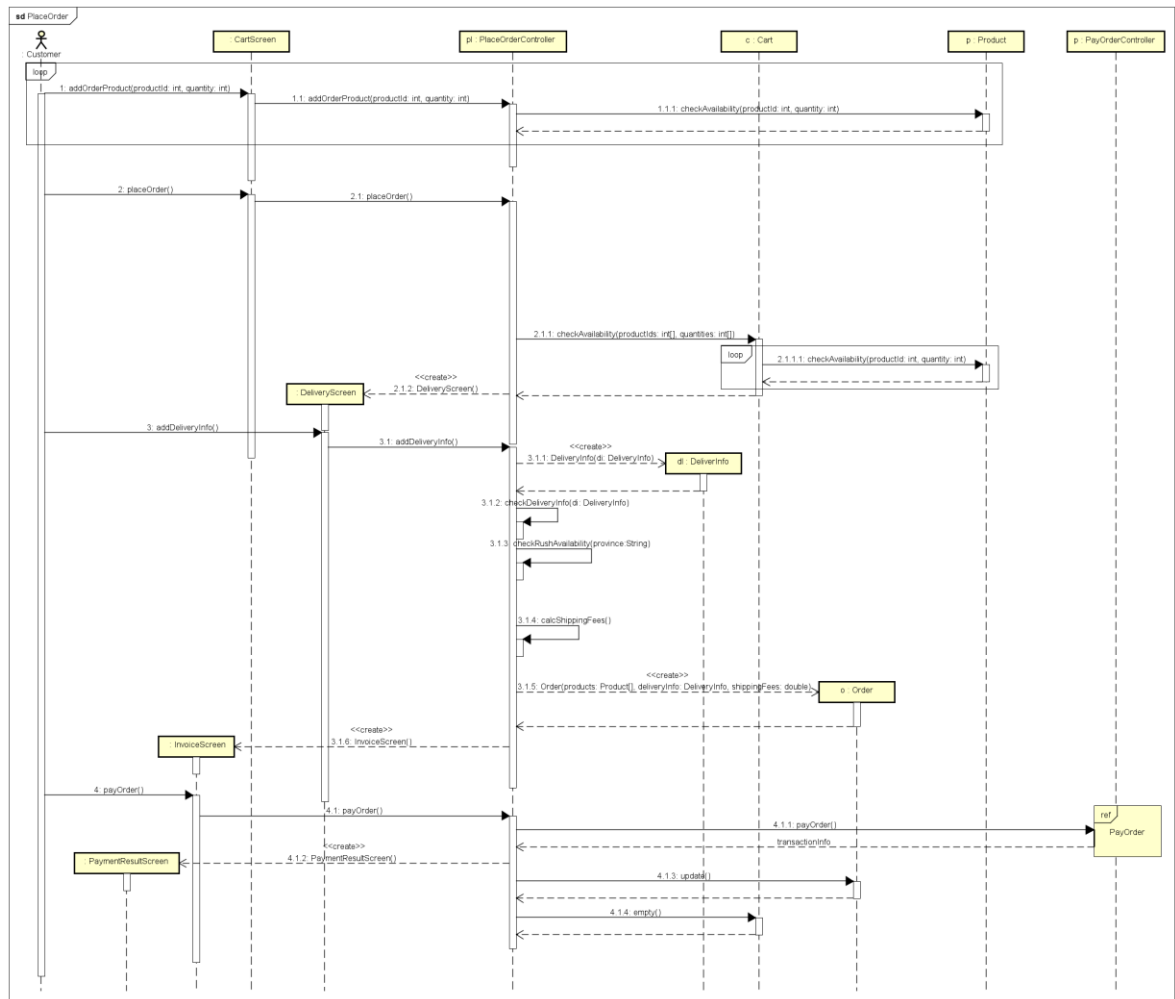
Chương 2: Phân tích thiết kế

Các thiết kế chi tiết được miêu tả trong tài liệu thiết kế (SDD) của nhóm. Sau đây là các nội dung nổi bật nhất trong quá trình thiết kế.

1. Các chức năng chính

Chức năng chính được chọn để phân tích thiết kế cho hệ thống bao gồm: Place order – Đặt hàng, và Pay order – Thanh toán đơn hàng

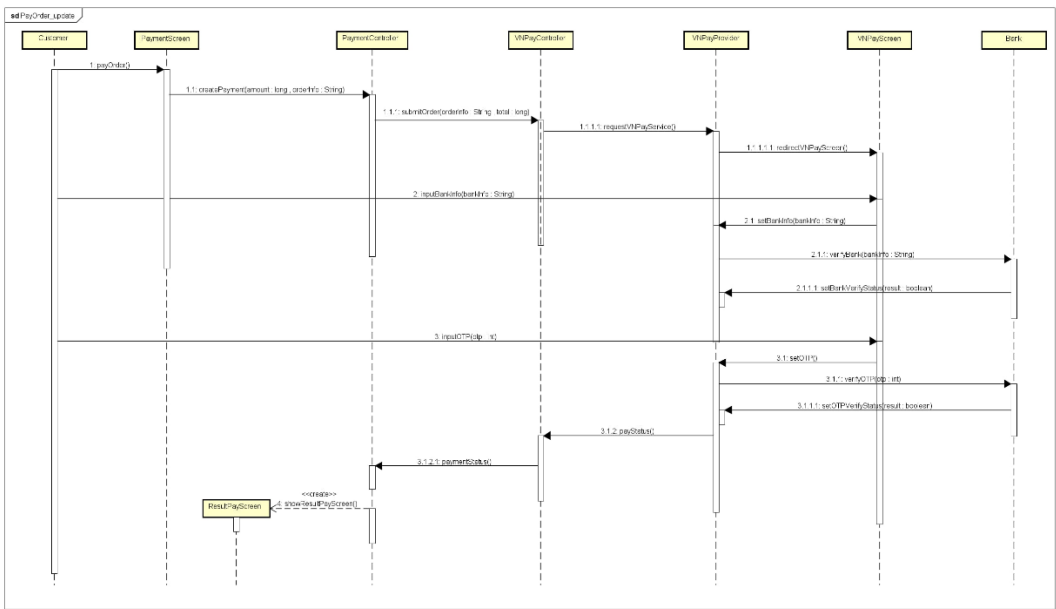
1.1. Use case Place order – Đặt hàng



Hình 2. Biểu đồ tuần tự use case Place order - đặt hàng

1.2. Use case Pay order – Thanh toán đơn hàng

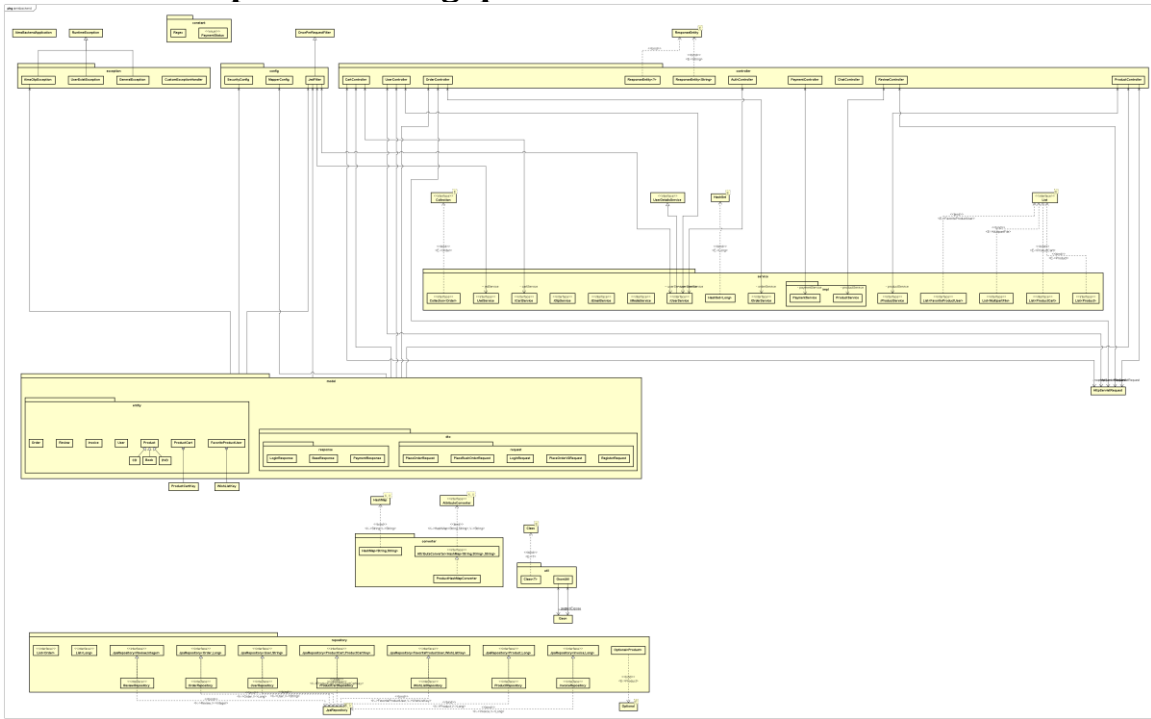
Thiết kế và xây dựng phần mềm



Hình 3. Biểu đồ tuần tự use case Thanh toán đơn hàng

2. Biểu đồ lớp thiết kế

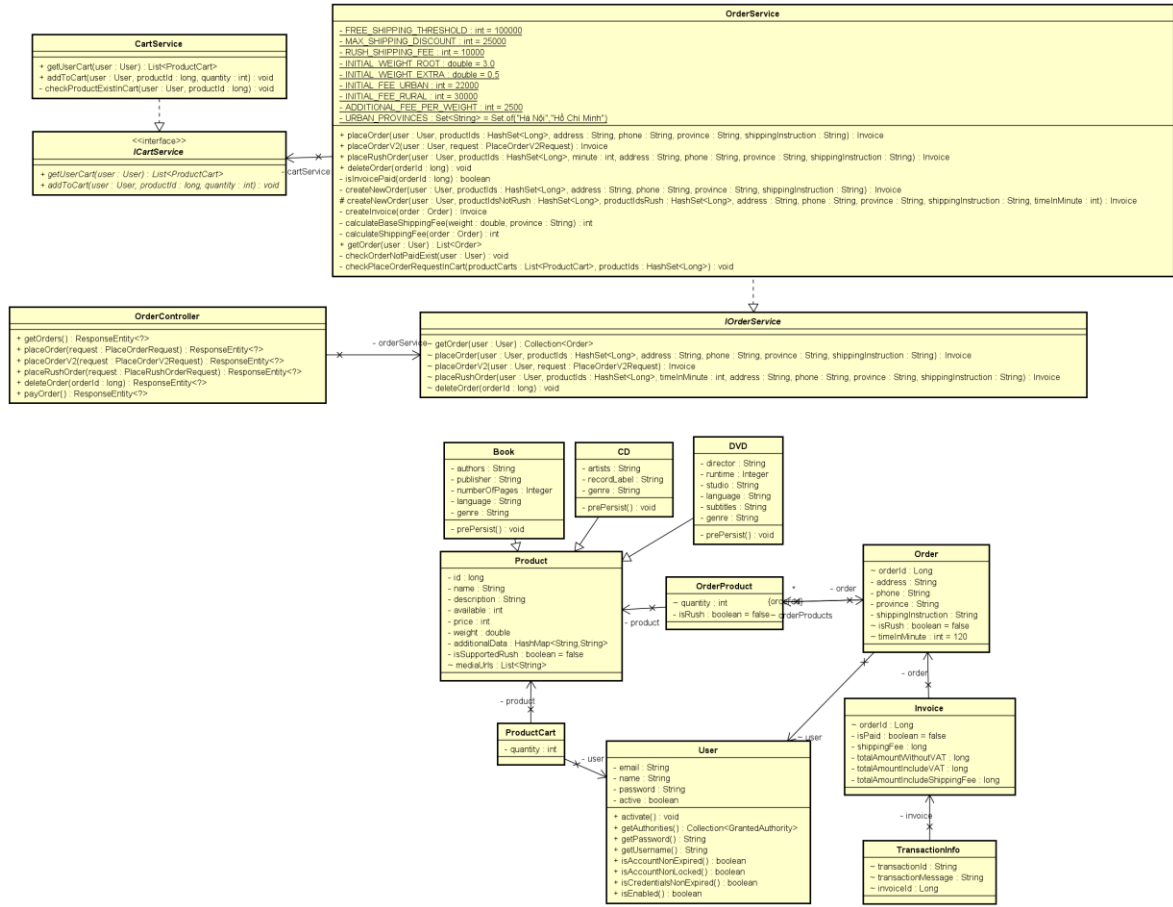
2.1. Biểu đồ lớp thiết kế tổng quan



Hình 4. Biểu đồ lớp thiết kế tổng quan

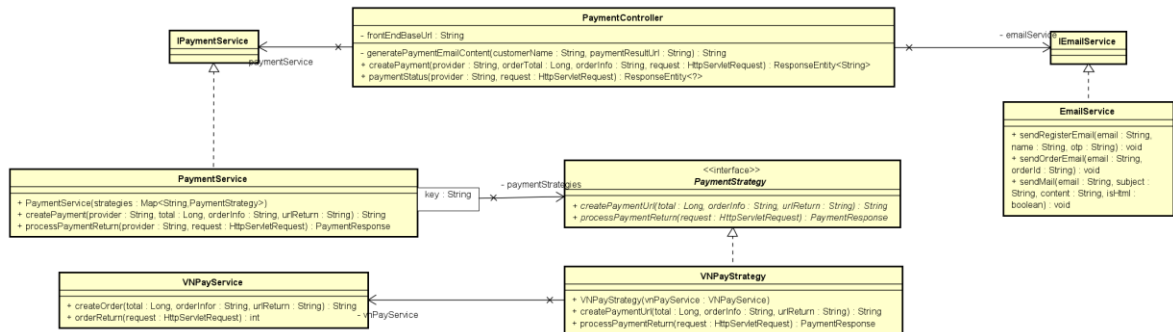
Thiết kế và xây dựng phần mềm

2.2. Biểu đồ lớp thiết kế cho use case đặt hàng



Hình 5. Biểu đồ lớp thiết kế cho use case Place order - đặt hàng

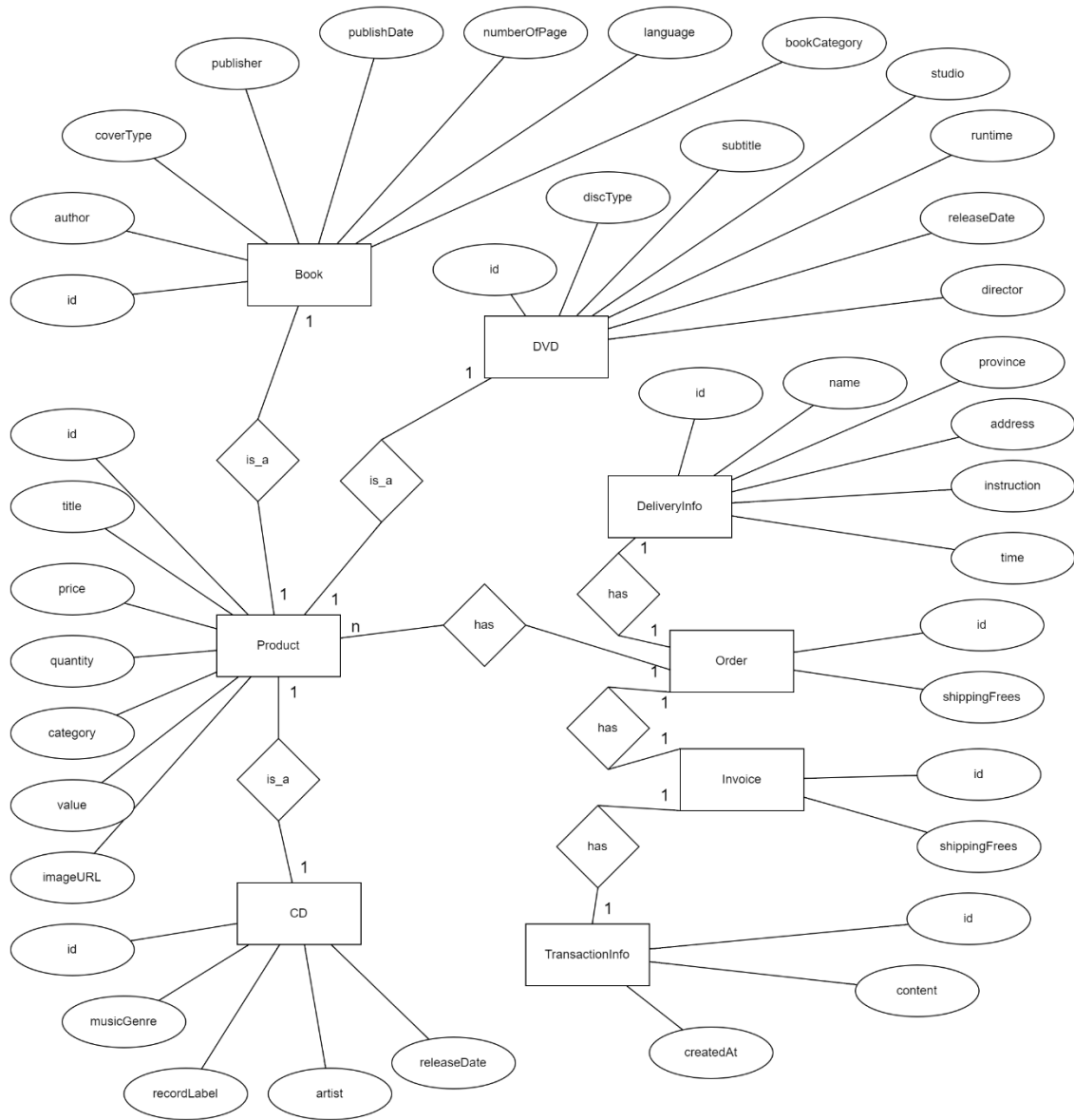
2.3. Biểu đồ lớp thiết kế cho hệ thống thanh toán



Hình 6. Biểu đồ lớp thiết kế cho use case Pay order - thanh toán đơn hàng

3. Thiết kế cơ sở dữ liệu

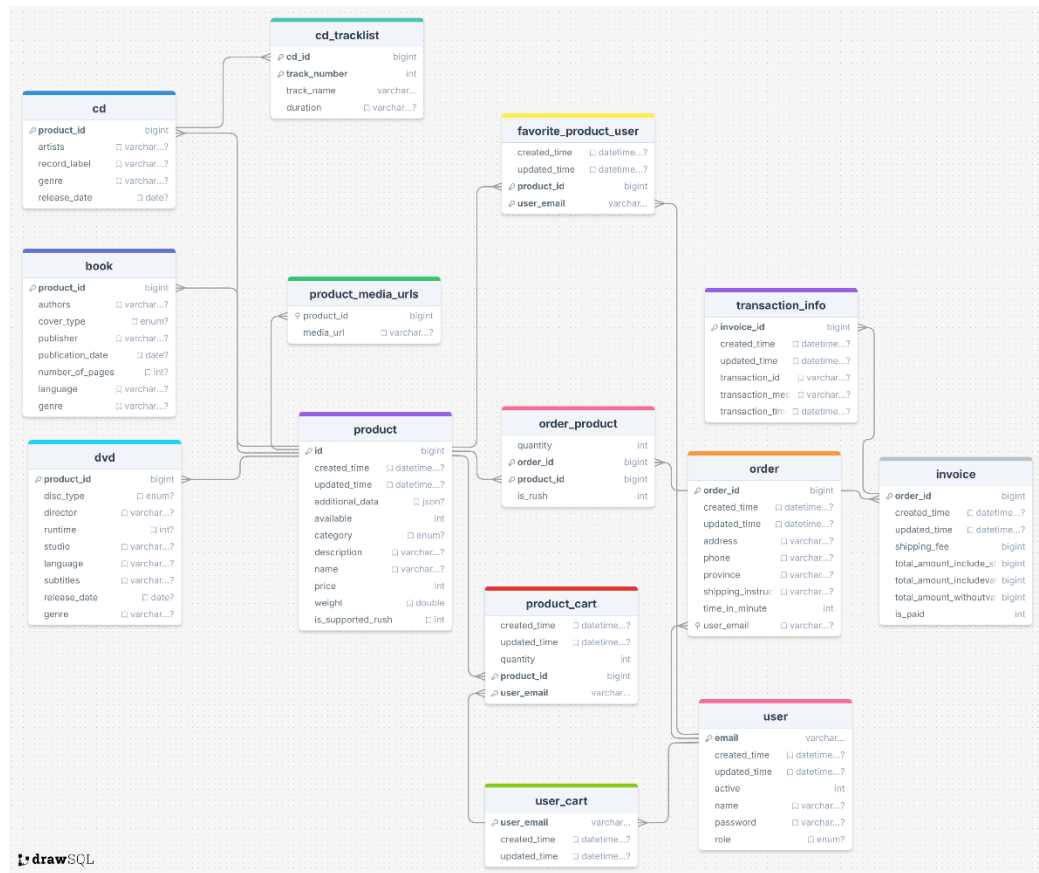
3.1. Phân tích các thực thể



Hình 7. Sơ đồ thực thể liên kết phân tích

3.2. Thiết kế cơ sở dữ liệu

Thiết kế và xây dựng phần mềm



Hình 8. Sơ đồ thực thể liên kết

Chi tiết xem thêm trong tài liệu thiết kế chi tiết (SDD)

Chương 3: Xây dựng

1. Xây dựng frontend bằng ReactJS

Trang web được phát triển dựa trên công nghệ ReactJS, một thư viện JavaScript mạnh mẽ cho phép xây dựng giao diện người dùng tương tác và hiệu quả. ReactJS giúp tạo ra các thành phần giao diện độc lập, dễ dàng quản lý và tái sử dụng. Ngoài ra, nhóm còn sử dụng các công cụ hỗ trợ như React Router để quản lý điều hướng. Với sự kết hợp của những công nghệ tiên tiến này, trang web không chỉ có giao diện đẹp mắt mà còn đảm bảo hiệu suất cao và trải nghiệm người dùng mượt mà.

2. Xây dựng backend bằng Spring Boot

Phần backend của trang Web được xây dựng bằng Spring Boot, một framework mạnh mẽ dựa trên Java. Spring Boot cho phép phát triển các ứng dụng web và API một cách nhanh chóng và hiệu quả nhờ vào cấu hình tự động và quản lý phụ thuộc mạnh mẽ. Với Spring Boot, nhóm có thể tạo ra các dịch vụ RESTful để giao tiếp với frontend một cách mượt mà.

Hơn nữa, Spring Boot tích hợp dễ dàng với các hệ thống cơ sở dữ liệu, và nhóm sử dụng Hibernate ORM để quản lý và truy vấn cơ sở dữ liệu. Điều này giúp đảm bảo rằng dữ liệu của trang web được xử lý một cách hiệu quả và nhất quán.

Spring Security cũng được tích hợp để đảm bảo an ninh cho ứng dụng, bảo vệ dữ liệu người dùng và ngăn chặn các cuộc tấn công phổ biến. Ngoài ra, việc sử dụng các công cụ như Spring Data JPA giúp đơn giản hóa việc truy cập và thao tác dữ liệu, tạo ra các API linh hoạt và dễ dàng mở rộng.

Với nền tảng vững chắc từ Spring Boot, backend của trang web đảm bảo được tính ổn định, bảo mật và hiệu suất cao, hỗ trợ tối đa cho trải nghiệm người dùng trên frontend.

3. Cơ sở dữ liệu MySQL

MySQL được sử dụng làm cơ sở dữ liệu vì các lý do sau:

Hiệu suất Cao và Khả Năng Mở Rộng: MySQL xử lý khối lượng lớn dữ liệu nhanh chóng và dễ dàng mở rộng khi cần.

Tính Bền Vững và Phục Hồi: Hỗ trợ giao dịch ACID và các công cụ sao lưu, phục hồi dữ liệu hiệu quả.

Bảo Mật: Cung cấp các tính năng bảo mật mạnh mẽ như xác thực người dùng và mã hóa dữ liệu.

Dễ Sử Dụng: Cung cấp giao diện quản lý thân thiện và hỗ trợ chuẩn SQL.

Tương Thích Đa Nền Tảng: Chạy trên nhiều hệ điều hành và hỗ trợ nhiều ngôn ngữ lập trình.

Thiết kế và xây dựng phần mềm

Cộng Đồng và Hỗ Trợ Mạnh Mẽ: Cộng đồng lớn, tài liệu phong phú, và hỗ trợ từ Oracle.

Chi Phí Thấp: Phiên bản cộng đồng miễn phí và tổng chi phí sở hữu thấp.

Những yếu tố này làm cho MySQL trở thành một lựa chọn lý tưởng cho các dự án cơ sở dữ liệu từ nhỏ đến lớn.

Thiết kế và xây dựng phần mềm

Bảng 3. Thiết kế test cases validateAddress()

Conditions	R1	R2	R3	R4
Only letters(a-z and A-z) or digits(0-9) or slashes	F	T	T	T
Number of characters	-	0	>0 && <=100	>100
Actions	validate	no	yes	no
Test Case Input				Expected Result
" Hello @ here!#1"				FALSE
""(null)				FALSE
"So 1 / Dai Co Viet"				TRUE
"This is such a long address that it should exceed one hundred characters already, if it has not, I am writing to fullfil at least one hundred and one characters 101/160"				FALSE

Bảng 4. Thiết kế test cases calcShippingFees()

Case	Order Value (VND)	Location	Weight (kg)	Rush Delivery	Expected Fee (VND)
R1	120,000	Hanoi	3.5	Yes	9,500
R2	90,000	HCM	3.5	Yes	34,500
R3	110,000	Hai Duong	3.5	Yes	30,000
R4	110,000	Hanoi	1.0	Yes	7,000
R5	110,000	HCM	3.5	No	0
R6	90,000	Hai Duong	2.0	Yes	47,500
R7	90,000	Hanoi	1.0	Yes	32,000
R8	90,000	HCM	3.5	No	24,500
R9	110,000	Hai Duong	1.5	No	10,000
R10	110,000	Hanoi	0.5	No	0
R11	110,000	Hai Duong	0.2	Yes	15,000
R12	90,000	Hai Duong	0.3	Yes	40,000
R13	90,000	Hai Duong	1.0	No	32,500
R14	90,000	Hanoi	1.0	No	22,000
R15	110,000	Hai Duong	0.3	No	5,000
R16	90,000	Hai Duong	0.2	No	30,000
Case	Value >100k VND	In Hanoi/HCM	Weight Exceeds Initial	Has Rush Item	Reduced Shipping Fee
R1	Yes	Yes	Yes	Yes	Yes
R2	No	Yes	Yes	Yes	No
R3	Yes	No	Yes	Yes	Yes
R4	Yes	Yes	No	Yes	Yes
R5	Yes	Yes	Yes	No	Yes
R6	No	No	Yes	Yes	No
R7	No	Yes	No	Yes	No

Thiết kế và xây dựng phần mềm

R8	No	Yes	Yes	No	No
R9	Yes	No	Yes	No	Yes
R10	Yes	Yes	No	No	Yes
R11	Yes	No	No	Yes	Yes
R12	No	No	No	Yes	No
R13	No	No	Yes	No	No
R14	No	Yes	No	No	No
R15	Yes	No	No	No	Yes
R16	No	No	No	No	No
Fee Type		Amount (VND)			
Initial 3kg cost		22,000			
Initial 0.5kg cost		30,000			
Additional per 0.5kg		2,500			
Rush delivery (per item)		10,000			
Shipping fee reduction		25,000			

2. Kết quả kiểm thử

Các test case thiết kế khi chạy JUnit đã đạt toàn bộ các test cases và test suite

✓ JUnitNameTest (isd.aims.main.controller)	89 ms
✓ validateNameTest(String, boolean)	89 ms
✓ [1] null, false	86 ms
✓ [2] TrungChien, true	1 ms
✓ [3] Hell0 Me!+ , false	1 ms
✓ [4] ThisIsSuchALongNameThatItWouldNotBeAppropriate, false	1 ms

Hình 9. Kết quả test 01

✓ JUnitPhoneNumberTest (isd.aims.main.controller)	97 ms
✓ validatePhoneNumber(String, boolean)	97 ms
✓ [1] abc123@! - , false	86 ms
✓ [2] 012-3, false	1 ms
✓ [3] 1324, false	1 ms
✓ [4] 0989123456, true	1 ms
✓ [5] 0123.123-233, false	2 ms
✓ [6] 0912--923-211, false	5 ms
✓ [7] 0989/123/456, true	1 ms

Hình 10. Kết quả test 02

Thiết kế và xây dựng phần mềm

JUnitDeliveryInfoTest (isd.aims.main.controller)	1 ms
JUnit Jupiter	0 ms
JUnitNameTest	0 ms
JUnitPhoneNumberTest	0 ms
JUnitAddressTest	0 ms
String, boolean)	1 ms
[1] TrungChien, false(testNameValidation	0 ms
[2] null, true(testNameValidation	0 ms
[3] HellO Me!+ , true(testNameValidation	0 ms
[4] ThisIsSuchALongNameThatItWouldNotBeAppropriate, true(testName	0 ms
[1] null, false(validateNameTest	0 ms
[2] TrungChien, true(validateNameTest	0 ms
[3] HellO Me!+ , false(validateNameTest	0 ms
[4] ThisIsSuchALongNameThatItWouldNotBeAppropriate, false(validateN	0 ms
[1] abc123@! - , false(validatePhoneNumber	0 ms
[2] 012-3, false(validatePhoneNumber	0 ms
[3] 1324, false(validatePhoneNumber	0 ms
[4] 0989123456, true(validatePhoneNumber	0 ms
[5] 0123.123-233, false(validatePhoneNumber	0 ms
[6] 0912--923-211, false(validatePhoneNumber	0 ms
[7] 0989/123/456, true(validatePhoneNumber	0 ms
[1] 0989123456, false(testPhoneNumberValidation	0 ms
[2] , true(testPhoneNumberValidation	1 ms
[3] 012-3, true(testPhoneNumberValidation	0 ms
[4] 1324, true(testPhoneNumberValidation	0 ms
[5] 0123.123-233, true(testPhoneNumberValidation	0 ms
[6] 0912--923-211, true(testPhoneNumberValidation	0 ms
[7] 0989/123/456, false(testPhoneNumberValidation	0 ms
[1] So 1 / Dai Co Viet, false(testAddressValidation	0 ms
[2] Hello @ here!#1, true(testAddressValidation	0 ms
[3] null, true(testAddressValidation	0 ms
[4] This is such a long address that it should exceed one hundred chara	0 ms
[1] Hello @ here!#1, false(validAddressTest	0 ms
[2] null, false(validAddressTest	0 ms
[3] So 1 / Dai Co Viet, true(validAddressTest	0 ms
[4] This is such a long address that it should exceed one hundred chara	0 ms

Hình 11. Kết quả test 03

Thiết kế và xây dựng phần mềm

✓ JUnitCalcShippingFeeTest (isd.aims.main.controller)	119 ms
✓ calculateShippingFeeTest(int, String, Double, int, int)	119 ms
✓ [1] 120000, Hanoi, 3.5, 1, 9500	105 ms
✓ [2] 90000, HCM, 3.5, 1, 34500	2 ms
✓ [3] 110000, Hai Duong, 3.5, 1, 30000	1 ms
✓ [4] 110000, Hanoi, 1.0, 1, 7000	1 ms
✓ [5] 110000, HCM, 3.5, 0, 0	1 ms
✓ [6] 90000, Hai Duong, 2.0, 1, 47500	1 ms
✓ [7] 90000, Hanoi, 1.0, 1, 32000	1 ms
✓ [8] 90000, HCM, 3.5, 0, 24500	1 ms
✓ [9] 110000, Hai Duong, 1.5, 0, 10000	1 ms
✓ [10] 110000, Hanoi, 0.5, 0, 0	1 ms
✓ [11] 110000, Hai Duong, 0.2, 1, 15000	
✓ [12] 90000, Hai Duong, 0.3, 1, 40000	
✓ [13] 90000, Hai Duong, 1.0, 0, 32500	1 ms
✓ [14] 90000, Hanoi, 1.0, 0, 22000	1 ms
✓ [15] 110000, Hai Duong, 0.3, 0, 5000	1 ms
✓ [16] 90000, Hai Duong, 0.2, 0, 30000	1 ms

Hình 12. Kết quả test 04

Chương 5: Đánh giá

1. Cohesion, coupling

1.1. Coupling

Sau khi đã thiết kế và triển khai, sửa đổi theo các tuần, mã nguồn của dự án đã đạt được mức lý tưởng – data coupling ở hầu hết các thành phần của hệ thống. Tuy nhiên, các lớp config phụ thuộc vào thiết kế của Framework chấp nhận duy trì tình trạng hiện tại: common coupling vì lý do sử dụng cấu hình cho toàn bộ hệ thống.

Các lỗi đã từng gặp tại hệ thống:

Bảng 5. Common coupling

Mô đun liên quan	Mô tả	Cải thiện
SecurityConfig	Đang sử dụng một instance singleton tĩnh WHITE_LIST, được chia sẻ với JwtFilter	Loại bỏ WHITE_LIST khỏi JwtFilter và định nghĩa một phương thức kiểm tra trong security config để xác minh xem route có được chấp nhận hay không. Cũng có thể giữ nguyên, tôn trọng cách thiết kế của Framework
Regex	Sử dụng các biến tĩnh để chứa các biểu thức chính quy để kiểm tra đầu vào	Giữ nguyên như hiện tại vì đây là một lớp tiện ích

Bảng 6. Stamp coupling

Module liên quan	Mô tả	Cải thiện
CartService	addToCart và checkProductExistIncart đang nhận toàn bộ đối tượng User làm tham số, nhưng chỉ cần email người dùng	Chỉ truyền String email thay vì User user
JwtService	generateToken nhận toàn bộ User userDetails nhưng chỉ sử dụng thuộc tính role và username	Chỉ truyền String userRole và String username thay vì User userDetails

Thiết kế và xây dựng phần mềm

ProductService	getWishList đang nhận toàn bộ User nhưng chỉ cần email người dùng	Chỉ truyền String userEmail thay vì User
----------------	---	--

Các vấn đề đã được giải quyết và toàn bộ các modules đã đạt mức Data coupling.

1.2. Cohesion

Do xây dựng lại từ đầu nên các mô đun được thiết kế và triển khai đạt mức đánh giá tối ưu về cohesion: Functional cohesion, không bị chồng chéo trách nhiệm giữa các mô đun

2. Nguyên tắc thiết kế SOLID

Những thiết kế ban đầu, có một số vấn đề khi áp dụng các nguyên tắc SOLID để đánh giá chất lượng.

2.1. Single Responsibility Principle

Về nguyên tắc Trách nhiệm Đơn lẻ (Single Responsibility Principle), tất cả các module đều đạt tiêu chuẩn vì không tìm thấy vấn đề về tính gắn kết.

2.2. Open-Closed Principle

Về nguyên tắc Đóng-Mở (Open-Closed Principle), module SecurityConfig đang có vấn đề về liên kết chung (Common coupling) có thể vi phạm nguyên tắc này, khi có thêm route mới trong WHITE_LIST, các lập trình viên phải sửa đổi module. Module Regex cũng có vấn đề về liên kết chung vi phạm nguyên tắc này nhưng với tư cách là một module cấu hình. Tuy nhiên, các lớp được thiết kế để có thể cấu hình và mở rộng thông qua cơ chế cấu hình của Spring, như định nghĩa beans và sử dụng phương thức ghi đè. Một số module service như CartService, JwtService, ProductService đang có vấn đề về liên kết tem (Stamp coupling), điều này có thể dẫn đến vi phạm các nguyên tắc nếu có bất kỳ thay đổi nào trong cấu trúc của lớp User. Về cải thiện, một số hướng đã được đề cập trong báo cáo đánh giá Cohesion và Coupling, và với những thay đổi đó, các vấn đề này phần lớn đã được giải quyết.

Việc phát triển mở rộng ứng dụng (thêm hình thức thanh toán, thêm loại sản phẩm bán, hoặc thay đổi cách tính phí vận chuyển) từ đó cũng trở nên thuận tiện hơn. Cụ thể, đội triển khai có thể thêm các lớp kế thừa và thực thi các interface, mở rộng mà không cần chỉnh sửa mã nguồn hiện tại

2.3. Liskov Substitution Principle

Về nguyên tắc Thay thế Liskov (Liskov Substitution Principle), một số lớp cơ sở tùy chỉnh như BaseEntity, BaseResponse được định nghĩa cho việc tổng quát hóa và các lớp kế thừa của nó có thể thay thế hoàn toàn cho lớp cha. Tất cả các lớp Entity

Thiết kế và xây dựng phần mềm

không xung đột với BaseEntity, còn về response, ở giai đoạn này chưa có gì để đánh giá. Cây kế thừa đều là 2 thế hệ và chưa phát hiện xung đột.

2.4. Interface Segregation Principle

Các interface được định nghĩa là các lớp có hậu tố 'Repository', mỗi interface phục vụ một chức năng nhất định mà không có quá nhiều phương thức trong một interface. Đã có các interface để quản lý Order, Product, ProductCart, User, WishList và Review (hai cái sau là chức năng mở rộng mà nhóm có thể thiết kế và triển khai). Do đó, không có vấn đề với nguyên tắc Phân tách Interface (Interface Segregation Principle).

2.5. Dependency Inversion Principle

Cuối cùng nhưng không kém phần quan trọng, trước thiết kế hiện tại, cấu trúc giao tiếp đi từ: Controller – Service – Repository, cấp cao – cấp thấp – repository, và việc triển khai hiện tại cho thấy controller phụ thuộc trực tiếp vào service, điều này vi phạm nguyên tắc. Hướng cải thiện là tổng quát hóa một interface cho controller – service, để service triển khai interface đó và controller sẽ phụ thuộc vào interface. Sau khi sửa đổi, các vấn đề đã được giải quyết.

3. Nguyên tắc thiết kế GRASP

3.1. Information Expert

- Principle: Giao trách nhiệm cho lớp có nhiều thông tin nhất cần thiết để thực hiện trách nhiệm đó.

- Analysis:

- `UserService`: Xử lý các hoạt động liên quan đến người dùng như đăng ký, đăng nhập và xác minh. Nó có quyền truy cập vào `UserRepository` và các dịch vụ khác như `OtpService` và `EmailService`, khiến nó trở thành chuyên gia thông tin cho việc quản lý người dùng.

- `OrderService`: Quản lý các hoạt động liên quan đến đơn hàng, bao gồm đặt hàng và tính toán phí vận chuyển. Nó có quyền truy cập vào `OrderRepository`, `CartService` và `InvoiceRepository`, khiến nó trở thành chuyên gia cho việc quản lý đơn hàng.

- `ProductService`: Xử lý các hoạt động liên quan đến sản phẩm, bao gồm đánh giá và danh sách yêu thích. Nó có quyền truy cập vào `ProductRepository`, `ReviewRepository` và `WishListRepository`, khiến nó trở thành chuyên gia cho việc quản lý sản phẩm.

- Improvement: Một số trách nhiệm có thể được ủy quyền thêm. Ví dụ, `OrderService` có thể ủy quyền việc tính toán phí vận chuyển cho một `ShippingService` riêng để tuân thủ nghiêm ngặt hơn Nguyên tắc Trách nhiệm Đơn nhất (SRP).

Thiết kế và xây dựng phần mềm

3.2. Creator

- Principle: Giao trách nhiệm tạo một đối tượng cho lớp chứa hoặc tổng hợp đối tượng đó.
- Analysis:
 - `UserService`: Tạo các đối tượng `User` trong quá trình đăng ký và kích hoạt chúng trong quá trình xác minh.
 - `OrderService`: Tạo các đối tượng `Order` và `Invoice` khi đặt hàng.
 - `ProductService`: Tạo các đối tượng `Review` và `FavoriteProductUser` khi thêm đánh giá hoặc mục vào danh sách yêu thích.
- Improvement: Việc tạo đối tượng được giao tốt cho các lớp quản lý chúng. Không có vấn đề lớn ở đây.

3.3. Controller

- Principle: Giao trách nhiệm xử lý các sự kiện hệ thống cho một lớp đại diện cho toàn bộ hệ thống hoặc một kịch bản sử dụng.
- Analysis:
 - Các bộ điều khiển (`AuthController`, `CartController`, `OrderController`, v.v.): Các lớp này đóng vai trò là bộ điều khiển, xử lý các yêu cầu HTTP và ủy quyền logic nghiệp vụ cho các dịch vụ như `UserService`, `OrderService` và `ProductService`.
 - `JwtFilter`: Đóng vai trò là bộ điều khiển cho việc xác thực, chặn các yêu cầu và xác thực token JWT.
- Improvement: Các bộ điều khiển được thiết kế tốt và tập trung vào việc xử lý các yêu cầu HTTP.

3.4. Low Coupling

- Principle: Giao trách nhiệm để giảm thiểu sự phụ thuộc giữa các lớp.
- Analysis:
 - Services: Các dịch vụ: Các dịch vụ như `UserService`, `OrderService` và `ProductService` có low coupling, tương tác chủ yếu thông qua các kho lưu trữ và các dịch vụ khác.
 - Controllers: Các bộ điều khiển có liên kết chặt chẽ với các dịch vụ, điều này là chấp nhận được vì chúng đóng vai trò là điểm vào cho logic nghiệp vụ.
 - `JwtFilter`: Liên kết với `JwtService` và `UserService`, nhưng điều này là cần thiết cho việc xác thực.
- Improvement: Một số dịch vụ (ví dụ: `OrderService`) có nhiều trách nhiệm, điều này làm tăng sự liên kết. Cân nhắc chia nhỏ `OrderService` thành các dịch vụ nhỏ hơn, tập trung hơn

Thiết kế và xây dựng phần mềm

3.5. High Cohesion

- Principle: Giao trách nhiệm để giữ cho các lớp tập trung và dễ bảo trì.
- Analysis:
 - Services: Hầu hết các dịch vụ có tính gắn kết cao, tập trung vào một lĩnh vực cụ thể (ví dụ: `UserService` cho quản lý người dùng, `ProductService` cho quản lý sản phẩm).
 - Controllers: Các bộ điều khiển có tính gắn kết, xử lý các loại yêu cầu cụ thể
 - Improvement: Một số dịch vụ (ví dụ: `OrderService`) xử lý nhiều trách nhiệm (ví dụ: đặt hàng, tính toán phí vận chuyển). Chia nhỏ các dịch vụ này sẽ cải thiện tính gắn kết.

3.6. Polymorphism

- Principle: Sử dụng đa hình để xử lý các biến thể trong hành vi.
- Analysis:
 - `User`: Sử dụng enum `Role` để phân biệt giữa các vai trò người dùng (ví dụ: `CUSTOMER`, `ADMIN`). Điều này có thể được mở rộng để hỗ trợ hành vi đa hình nếu cần.
 - `PlaceRushOrderRequest`: Mở rộng `PlaceOrderRequest`, thể hiện tính kế thừa và đa hình.
 - Improvement: Đa hình chưa được sử dụng triệt để. Ví dụ, các loại đơn hàng khác nhau (ví dụ: đơn hàng gấp, đơn hàng tiêu chuẩn) có thể được mô hình hóa bằng đa hình.

3.7. Pure Fabrication

- Principle: Tạo các lớp nhân tạo để xử lý các trách nhiệm không thuộc về bất kỳ lớp hiện có nào.
- Analysis:
 - `JwtService`: Một lớp tạo thuần túy xử lý việc tạo và xác thực token JWT.
 - `OtpService`: Một lớp tạo thuần túy xử lý việc tạo và xác thực OTP.
 - Improvement: Không có vấn đề lớn ở đây. Các lớp này được thiết kế tốt và tập trung vào các trách nhiệm cụ thể của chúng.

3.8. Indirection

- Principle: Sử dụng các lớp trung gian để giảm sự liên kết giữa các thành phần.
- Analysis:
 - `JwtFilter`: Đóng vai trò là trung gian giữa yêu cầu HTTP và quá trình xác thực, giảm sự liên kết giữa các bộ điều khiển và logic xác thực.

Thiết kế và xây dựng phần mềm

- `ModelMapper`: Được sử dụng trong các bộ điều khiển để ánh xạ các thực thể sang DTO, giảm sự liên kết giữa các bộ điều khiển và các thực thể.

- Improvement: Không có vấn đề lớn ở đây. Gián tiếp được sử dụng hiệu quả để tách rời các thành phần.

3.9. Protected Variations

- Principle: Thiết kế hệ thống để ổn định trước các thay đổi bằng cách đóng gói các biến thể.

- Analysis:

- `JwtFilter`: Đóng gói logic xác thực, bảo vệ phần còn lại của hệ thống khỏi các thay đổi trong việc xử lý JWT.

- `OtpService`: Đóng gói việc tạo và xác thực OTP, bảo vệ hệ thống khỏi các thay đổi trong logic OTP.

- Improvement: Một số khu vực, như tính toán phí vận chuyển trong `OrderService`, không được bảo vệ tốt khỏi các biến thể. Cần nhắc đóng gói logic này trong một `ShippingService` riêng biệt.

3.10. General Observations

1. Tách biệt mối quan tâm: Dự án nói chung tuân thủ tốt việc tách biệt mối quan tâm, với các bộ điều khiển xử lý yêu cầu HTTP, các dịch vụ xử lý logic nghiệp vụ và các kho lưu trữ xử lý truy cập dữ liệu.

2. Nguyên tắc Trách nhiệm Đơn nhất (SRP): Một số dịch vụ (ví dụ: `OrderService`) xử lý nhiều trách nhiệm, có thể được chia nhỏ thành các dịch vụ nhỏ hơn, tập trung hơn.

3. Xử lý lỗi: Xử lý lỗi được tập trung trong `CustomExceptionHandler`, đây là thực hành tốt. Tuy nhiên, một số ngoại lệ (ví dụ: `GeneralException`) quá chung chung và có thể được thay thế bằng các ngoại lệ cụ thể hơn.

4. Khả năng tái sử dụng mã: Một số lớp tiện ích (ví dụ: `GsonUtil`) được thiết kế tốt và có thể tái sử dụng. Tuy nhiên, một số logic (ví dụ: tính toán phí vận chuyển) bị trùng lặp và có thể được tái cấu trúc thành các thành phần có thể tái sử dụng.

3.11. Recommendations

1. Tái cấu trúc `OrderService`: Chia nhỏ nó thành các dịch vụ nhỏ hơn (ví dụ: `ShippingService`, `InvoiceService`) để cải thiện tính gắn kết và giảm sự liên kết.

2. Sử dụng Đa hình: Mô hình hóa các loại đơn hàng khác nhau (ví dụ: đơn hàng gấp, đơn hàng tiêu chuẩn) bằng đa hình thay vì sử dụng các cờ.

3. Cải thiện Xử lý Lỗi: Thay thế các ngoại lệ chung chung (ví dụ: `GeneralException`) bằng các ngoại lệ cụ thể hơn để cải thiện việc xử lý lỗi.

4. Loại bỏ các Bộ điều khiển Trống: Loại bỏ hoặc triển khai các bộ điều khiển trống như `ChatController`.

Thiết kế và xây dựng phần mềm

5. Đóng gói Logic Vận chuyển: Di chuyển logic tính toán phí vận chuyển sang một `ShippingService` riêng biệt để bảo vệ khỏi các biến thể..

Chương 6: Mẫu thiết kế Design Pattern

1. Singleton Pattern

Mẫu Singleton đảm bảo rằng một lớp chỉ có một thể hiện (instance) và cung cấp một điểm truy cập toàn cục đến nó. Trong Spring, các bean được định nghĩa với chú thích @Bean mặc định là singleton. Ví dụ:

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

2. Factory Method Pattern

Mẫu Factory Method định nghĩa một giao diện để tạo đối tượng nhưng cho phép các lớp con thay đổi loại đối tượng sẽ được tạo. Trong Spring, chú thích @Bean có thể được xem như một phương thức factory.

3. Template Method Pattern

Mẫu Template Method định nghĩa khung của một thuật toán trong một phương thức, hoãn một số bước cho các lớp con. Các lớp cấu hình của Spring Security thường sử dụng mẫu này.

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {
    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http, JwtFilter
jwtFilter) throws Exception {
        http
            .cors(AbstractHttpConfigurer::disable)
            .csrf(AbstractHttpConfigurer::disable)
            .authorizeHttpRequests(auth -> {
                auth.requestMatchers("/api/auth/**").permitAll();
                auth.anyRequest().authenticated();
            })
            .sessionManagement(session
session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
    }
}
```

->

Thiết kế và xây dựng phần mềm

```
        .exceptionHandling(exception -> exception
            .authenticationEntryPoint((request, response, authException) ->
response.sendError(HttpServletResponse.SC_UNAUTHORIZED))
        )
        .addFilterBefore(jwtFilter,
UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }
}
```

4. Adapter Pattern

Mẫu Adapter cho phép các giao diện không tương thích làm việc cùng nhau. Trong Spring, ModelMapper được sử dụng để ánh xạ giữa các mô hình đối tượng khác nhau. Ví dụ:

```
@GetMapping("/info")
public ResponseEntity<?> getInfo() {
    return BaseResponse.ok(modelMapper.map(request.getAttribute("user"),
UserDto.class));
}
```

5. Builder Pattern

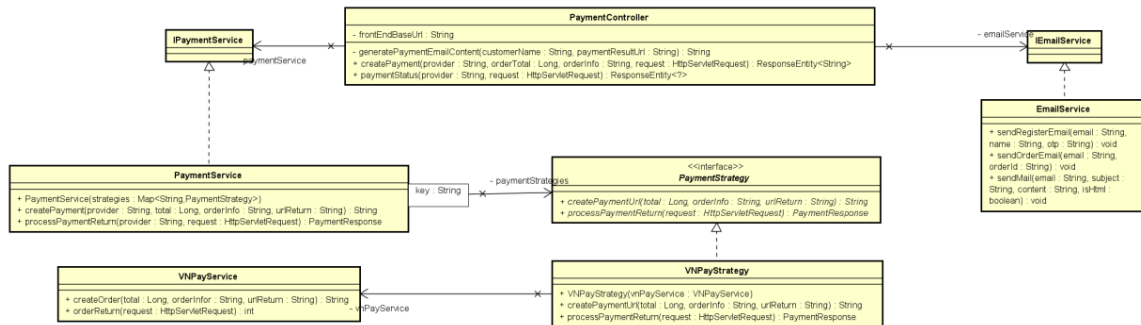
Mẫu Builder được sử dụng để xây dựng các đối tượng phức tạp theo từng bước. Cấu hình HttpSecurity trong Spring Security có thể được xem như một ví dụ của mẫu Builder.

```
http
    .cors(AbstractHttpConfigurer::disable)
    .csrf(AbstractHttpConfigurer::disable)
    .authorizeHttpRequests(auth -> {
        auth.requestMatchers("/api/auth/**").permitAll();
        auth.anyRequest().authenticated();
    })
    .sessionManagement(sessionManagement -> {
session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
    .exceptionHandling(exception -> exception
```

Thiết kế và xây dựng phần mềm

```
.authenticationEntryPoint((request, response, authException) ->
response.sendError(HttpServletResponse.SC_UNAUTHORIZED))
)
.addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);
```

6. Strategy Pattern



Hình 13. Strategy pattern

Xử lý thanh toán được triển khai với mẫu strategy đảm bảo khả năng mở rộng và linh hoạt của các phương thức thanh toán được tích hợp vào hệ thống.

Chương 7: Kết luận và hướng phát triển

Trong quá trình hoạt động, trang web AIMS em đã đạt được một số thành tựu cơ bản và đối mặt với một số thách thức cụ thể. Trang web đã thành công trong việc cung cấp nền tảng bán hàng trực tuyến hiệu quả, cho phép quản lý các sản phẩm thiết bị điện tử một cách rõ ràng, minh bạch và chính xác.

Em đã đặt ra một số ưu điểm quan trọng của trang web, bao gồm khả năng quản lý thông tin sản phẩm, thông tin khách hàng, và các giao dịch mua bán. Điều này đã nâng cao khả năng quản lý sản phẩm và dịch vụ cho trang web, đảm bảo rằng ứng dụng đáp ứng mọi nhu cầu của khách hàng một cách tốt nhất.

Tuy nhiên, em cũng nhận thức về một số nhược điểm cần cải thiện. Việc sử dụng công nghệ mới học, mặc dù mang lại nhiều lợi ích, nhưng cũng đặt ra khó khăn cho em trong quá trình học hỏi và áp dụng. Em cũng còn thiếu kỹ năng chuyên môn nên cũng ảnh hưởng đến chất lượng của trang web và khả năng bảo trì. Trong quá trình phát triển trang web, nhiều bug xuất hiện tốn rất nhiều thời gian để tìm ra vấn đề và sửa sai.

Để phát triển hơn, em nghĩ mình có thể xem xét việc nghiên cứu và áp dụng các công nghệ khác để đảm bảo trang web luôn hiện đại và đáp ứng nhu cầu của khách hàng. Bên cạnh đó, em nghĩ việc xây dựng quy trình làm việc rõ ràng và thống nhất cũng sẽ hỗ trợ trong quá trình quản lý và phát triển hiệu quả sản phẩm của em.

Thiết kế và xây dựng phần mềm

TÀI LIỆU THAM KHẢO